

PROJECT REFERENCE DOCUMENT

MedShield-FL

A Unified Privacy-Preserving Federated Learning Framework with Homomorphic Encryption for Secure Healthcare AI

Training accurate medical AI models across hospitals — without ever moving patient data.

What this document covers

Project introduction, problem statement, objectives, how the system works, full system architecture, complete technology stack (frontend + backend), target users, development roadmap, and future scope — all explained in plain, easy-to-follow language.

Table of Contents

Table of Contents	2
1. Introduction.....	3
2. Problem Statement	3
3. Objectives	3
4. How the System Works (Simplified)	4
5. System Architecture	5
5.1 Central Server Side.....	5
5.2 Hospital Client Side	5
5.3 A Typical Training Round, Visually.....	5
6. Technology Stack.....	5
6.1 Frontend (User-Facing Side)	5
6.2 Backend (Engine Side).....	6
6.3 Full Stack Summary Table	8
7. Who Should Use This / Target Audience	8
8. Key Features and Innovations.....	9
9. Development Roadmap.....	9
10. Future Scope	9
11. Conclusion	10

1. Introduction

Hospitals and research institutions collect huge amounts of medical data every day — X-rays, MRI scans, patient records, and more. This data is extremely valuable because it can be used to train Artificial Intelligence (AI) models that help doctors detect diseases like cancer, lung infections, and brain disorders faster and more accurately.

But there is a big problem. Laws like HIPAA (in the USA) and GDPR (in Europe) do not allow hospitals to freely share patient data with each other or send it to a central company or server. This is for good reason — patient privacy must be protected. However, this also means that no single hospital has enough data on its own to train a really accurate, reliable AI model. A model trained on data from just one hospital tends to make mistakes when it sees patients or scanner types it has never seen before.

MedShield-FL solves this exact problem. Instead of moving patient data to one place, it moves the AI model to each hospital. Every hospital trains the model using its own local, private data. Only the learnings (called "model updates") are shared — and even these learnings are locked using strong encryption before they leave the hospital, so nobody, not even the central server, can ever see the real values. This way, all hospitals can work together to build one powerful, shared AI model, without a single patient record ever leaving its hospital.

In one line

MedShield-FL lets many hospitals train one shared, highly accurate medical AI model together — without any hospital ever sharing its actual patient data with anyone.

2. Problem Statement

Today, organizations building medical AI face a difficult contradiction that this project aims to resolve:

- AI models need large and varied datasets to be accurate — but hospitals hold data in small, isolated pockets.
- Privacy laws (HIPAA, GDPR) strictly prohibit centralizing raw patient data in one place.
- Models trained on a single hospital's small dataset tend to "overfit" — they work well on that hospital's patients but fail on others.
- Existing privacy solutions (like basic differential privacy) often reduce model accuracy, and encryption-heavy solutions are usually too slow or use too much internet bandwidth to be practical.
- Annotating (labelling) medical images needs expert doctors, and their time is limited and expensive — so wasting it on unhelpful images is costly.
- Doctors do not trust "black-box" AI decisions unless they can see and understand why the AI reached a conclusion.

MedShield-FL is designed to solve every one of these problems together, in a single, unified framework — rather than solving them one at a time with disconnected tools.

3. Objectives

The project has the following core objectives:

1. Enable collaborative training: Allow multiple hospitals to jointly train one shared AI model without moving or exposing raw patient data.
2. Guarantee data privacy mathematically: Use Homomorphic Encryption (CKKS scheme) so that even the central server can never view real patient data or even the plain model updates — it only ever touches encrypted values.
3. Reduce network/bandwidth cost: Use Vision Transformers (ViT) and encrypt only the most critical parts of the model (like the [CLS] token), cutting data transfer by up to 30 times compared to encrypting everything.
4. Make smart use of doctors' time: Use Active Learning so the system automatically picks only the most useful, uncertain images for doctors to label — instead of wasting effort on easy or irrelevant images.
5. Handle real-world data differences: Ensure the model still performs well even when different hospitals use different scanners, patient populations, or imaging protocols (this is called "domain shift").
6. Build clinical trust through explainability: Use Explainable AI (Grad-CAM) so doctors can see a visual heatmap of exactly which part of a scan influenced the AI's decision.

4. How the System Works (Simplified)

Rather than diving into heavy technical detail, here is the core idea of MedShield-FL explained step-by-step, the way it happens in every training cycle (called a "round"):

7. Start: The central server sends the current version of the shared AI model to every participating hospital.
8. Local training: Each hospital trains this model using only its own local patient images. The data never leaves the hospital's own computers.
9. Locking the results: Before sending anything back, the hospital encrypts (locks) its results using homomorphic encryption. This turns the numbers into unreadable, scrambled ciphertext.
10. Sending it over: The hospital sends only this encrypted (locked) result to the central server — never the raw data, and never even the unlocked model result.
11. Blind averaging: The central server combines (averages) the encrypted results from all hospitals. Because of the special math behind homomorphic encryption, it can do this correctly without ever unlocking or seeing the actual numbers.
12. Sending the update back: The newly combined (still encrypted) model is sent back to every hospital.
13. Unlocking locally: Each hospital uses its own private key (which never leaves that hospital) to unlock just its copy of the updated model.
14. Repeat: This cycle repeats many times, and with every round the shared model gets smarter — all while patient data stays completely private.

Why this matters

The central server is designed to be "blind." It can do useful math on the encrypted numbers, but it mathematically cannot see, guess, or reconstruct any patient's real data. This removes the need to "trust" the server — the privacy is guaranteed by mathematics, not just policy.

5. System Architecture

The system has two main sides that talk to each other over the internet: the Central Server (which coordinates everything) and the Hospital Client (which runs inside each hospital's own network).

5.1 Central Server Side

- **Orchestration Hub:** Manages which hospitals are connected and keeps every training round in sync.
- **Encrypted Aggregator:** Combines the locked results from all hospitals using homomorphic math, without ever unlocking them.
- **Global Model Store:** Safely holds the current (still-encrypted) version of the shared model, ready to send back out to hospitals.

5.2 Hospital Client Side

- **Data Preparation Module:** Cleans and prepares medical images (X-rays, MRI, etc.) so they are ready for the AI model.
- **Smart Labelling Assistant:** Looks at unlabeled images and flags only the most useful, uncertain ones for a doctor to review and label.
- **Local AI Training Engine:** Trains the shared model using the hospital's own labelled data.
- **Encryption Manager:** Locks the training results before they leave the hospital, and unlocks new updates when they arrive. The private key used for this never leaves the hospital.

5.3 A Typical Training Round, Visually

Step	What Happens	Where
1	Model is shared with all hospitals	Server → Hospitals
2	Doctors label only the most useful images	Inside Hospital
3	Model trains on local, private data	Inside Hospital
4	Results are encrypted (locked)	Inside Hospital
5	Locked results are sent over the network	Hospital → Server
6	Server blindly combines all locked results	At Server
7	Updated locked model sent back	Server → Hospitals
8	Each hospital unlocks its own copy	Inside Hospital

6. Technology Stack

This section explains what should be used to build MedShield-FL, split clearly into Frontend (what users see and interact with) and Backend (the engine running behind the scenes), plus the specialized AI/security layer that makes the project unique.

6.1 Frontend (User-Facing Side)

The frontend is the dashboard that hospital staff, administrators, and doctors will actually use. It should be simple, clean, and require no technical knowledge to operate. It does not touch any encryption or raw patient data directly — it only shows information and results.

Component	What It Does / Should Include
Framework	React.js (with TypeScript) for building a fast, responsive web dashboard.
UI Components	Tailwind CSS / Material UI for clean, professional, consistent design.
Charts & Graphs	Recharts or Chart.js to show training progress, accuracy trends, and participation stats over time.
Image / Heatmap Viewer	A viewer component to display medical scans side-by-side with Grad-CAM heatmaps, so doctors can see exactly what the AI focused on.
Authentication Screens	Secure login pages for hospital admins and doctors (role-based access, e.g., Admin vs. Radiologist).
Labelling Interface	A simple screen where doctors review AI-flagged images and submit labels/diagnoses.
Notifications	Real-time alerts (e.g., "Training round complete", "5 new images need your review").
State Management	Redux Toolkit or React Query for handling live data updates from the backend.

What the Frontend Should Show

- A hospital dashboard: current training round, model accuracy, how many images this hospital has contributed.
- A doctor's queue: images the AI has flagged as most important to label, with the ability to zoom, annotate, and submit.
- An explainability screen: original scan + AI heatmap overlay + short plain-language explanation.
- An admin panel: which hospitals are currently connected, encryption status, and overall system health.
- A privacy status indicator: a visible confirmation that data never left the hospital (e.g., "Data stayed local ✓").

6.2 Backend (Engine Side)

The backend is where all the real computation happens — training the AI, encrypting results, and coordinating hospitals. This is split into a few specialized layers.

Component	What It Does / Should Include
API Layer	FastAPI (Python) to serve the frontend, handle authentication, and manage hospital session data.
Federated Orchestration	Flower (flwr) Framework — manages communication between the central server and every hospital node using gRPC.

Component	What It Does / Should Include
AI / Machine Learning Engine	PyTorch with the timm library, used to build and train the Vision Transformer (ViT) model.
Encryption Engine	TenSEAL (built on Microsoft SEAL) — implements CKKS Homomorphic Encryption to lock and unlock model updates.
Active Learning Module	Custom Python logic implementing evidential uncertainty scoring, to pick the most useful images for labelling.
Explainability Engine	Grad-CAM (adapted for Vision Transformers) to generate heatmaps explaining AI decisions.
Database	PostgreSQL for storing metadata only — things like hospital IDs, training logs, accuracy history, and user accounts. Patient images/data are never stored centrally.
Message/Task Queue	Redis or Celery to manage background jobs like training rounds and notifications.
Authentication & Security	OAuth2 / JWT tokens for secure login, plus TLS encryption on all network traffic.
Deployment & Hosting	Docker containers, orchestrated with Kubernetes, so each hospital can run its own client reliably.

Why This Split Matters

Notice that the frontend never directly handles patient data or private encryption keys — it only displays results and progress. All the sensitive, heavy-lifting work (training, encrypting, aggregating) happens purely on the backend, inside each hospital's own secured environment. This separation itself is a privacy safeguard.

- ✓ Internationally accepted benchmark dataset.
- ✓ High-quality expert-labelled MRI scans.
- ✓ Data comes from multiple hospitals.
- ✓ Perfect for testing privacy-preserving Federated Learning systems.

Why BraTS is a Good Choice

****Vision Transformer (ViT-Base/16)**** is chosen because:

- It gives high accuracy on MRI image classification.
- It can learn small details from medical images.
- It works well with ****Grad-CAM**** to show heatmaps explaining the AI's decision.
- It supports token-level encryption, which reduces communication cost in Federated Learning.

Why We Use Vision Transformer (ViT)

****Step 1:**** Every hospital keeps its own BraTS MRI scans.

****Step 2:**** The central server sends the latest Vision Transformer (ViT-Base/16) model.

****Step 3:**** Each hospital trains the model using only its local MRI images.

****Step 4:**** The model updates are encrypted using ****CKKS Homomorphic Encryption**** through ****TenSEAL****.

****Step 5:**** The encrypted updates are sent to the central server.

****Step 6:**** The server combines all encrypted updates using ****Federated Averaging (FedAvg)****.

****Step 7:**** The improved global model is sent back to every hospital.

****Step 8:**** These steps repeat until the model becomes highly accurate.

How the AI Model is Trained

- ****Glioma**** – A common brain tumor that starts in the brain's support cells.
- ****Meningioma**** – A tumor that grows in the protective covering of the brain.
- ****Pituitary Tumor**** – A tumor that develops in the pituitary gland and can affect hormones.

Example Brain Disorders

****BraTS (Brain Tumor Segmentation/Classification)**** is one of the most trusted datasets for brain MRI research. It contains MRI scans collected from different hospitals along with expert-labelled tumor information. The dataset includes T1, T1ce, T2 and FLAIR MRI images.

Dataset Used

This project uses Brain MRI scans to detect brain tumors. Brain MRI data is very sensitive and cannot be shared easily between hospitals. That is why it is a perfect example for Federated Learning, where every hospital trains the AI model on its own data without sharing patient information.

6.2.1 Medical AI Model and Dataset

6.3 Full Stack Summary Table

Layer	Technology	Purpose
Frontend	React.js, TypeScript, Tailwind CSS	Dashboards, image/heatmap viewer, labelling screens
API Layer	FastAPI (Python)	Connects frontend to backend securely
Federated Orchestration	Flower (flwr)	Coordinates all hospitals during training
AI Engine	PyTorch + timm	Trains the Vision Transformer diagnostic model
Encryption Engine	TenSEAL (Microsoft SEAL / CKKS)	Locks and unlocks model updates
Active Learning	Custom Python (Evidential DL)	Selects the most useful images to label
Explainability	Grad-CAM	Generates visual heatmaps for trust and transparency
Database	PostgreSQL	Stores metadata, logs, accuracy history only
Deployment	Docker + Kubernetes	Reliable deployment at each hospital and the central server

7. Who Should Use This / Target Audience

MedShield-FL is designed for anyone who needs accurate medical AI without compromising patient privacy:

- **Hospitals and Clinics:** to collaboratively improve diagnostic AI tools without sharing patient records.
- **Medical Research Institutions:** to run large, multi-site studies while staying fully compliant with privacy laws.
- **Diagnostic Imaging Networks:** radiology and pathology chains that want consistent AI performance across all their centers.
- **Health-Tech Companies:** building commercial diagnostic AI products that need real, diverse clinical data to train on.
- **Government & Regulatory Health Bodies:** for public health AI initiatives that must meet strict data protection standards.
- **Radiologists and Doctors:** as end users who benefit from more accurate, explainable AI-assisted diagnosis tools.

- Students and Researchers: studying privacy-preserving machine learning, federated learning, or applied cryptography.

8. Key Features and Innovations

Feature	Plain-Language Explanation
Homomorphic Encryption (CKKS)	Locks the numbers so math can still be done on them, without ever unlocking them — like solving a puzzle through a sealed box.
Vision Transformer (ViT) with Token Isolation	Only encrypts the most critical, sensitive part of the model instead of everything, making data transfer up to 30x lighter.
Federated Active Learning	Automatically finds the most useful, uncertain images so doctors don't waste time labelling easy or repeated ones.
Domain Shift Handling	Keeps the model accurate even when hospitals use different scanners or have very different patient populations.
Explainable AI (Grad-CAM)	Shows a visual heatmap on every scan, so doctors can see and verify exactly what the AI is "looking at."
Zero Data Movement	Patient data never physically leaves the hospital at any point in the entire process.

9. Development Roadmap

The project is planned in four clear phases (sprints), each one building on top of the last:

Phase	Goal	Key Tasks
Sprint 1 Foundation	Get the basic network of hospitals and server talking to each other.	Set up the Flower server and hospital client connections Test with a simple AI model (no encryption yet) Set up basic medical image data pipelines
Sprint 2 Encryption	Add the privacy/security layer.	Integrate TenSEAL encryption into the training loop Generate and manage encryption keys Measure and optimize the extra time encryption adds
Sprint 3 Efficiency	Make the system fast and lightweight enough for real hospitals.	Switch to the Vision Transformer model Encrypt only the critical [CLS] tokens Measure and confirm bandwidth savings
Sprint 4 Trust & Intelligence	Make the system smart and trustworthy for doctors.	Add smart image-selection (active learning) Add Grad-CAM explainability heatmaps Run full end-to-end testing

10. Future Scope

Once the core system is working, there is room to make MedShield-FL even stronger and more production-ready:

15. Defense against bad actors: Add checks (Zero-Knowledge Proofs) so the server can catch a hospital sending fake or corrupted updates — without ever having to decrypt and see the data.

16. Extra privacy layer: Add Differential Privacy (controlled random "noise") on top of encryption, so the final trained model can never accidentally memorize a specific patient's details.
17. Faster hardware support: Use GPU acceleration to speed up the currently slow encrypted computations, making the whole system faster for hospitals with limited hardware.
18. Lightweight edge deployment: Optimize the system so it can even run efficiently in small clinics with very basic computer hardware.
19. Wider disease coverage: Expand beyond the initial imaging types to cover more diseases and scan types (CT, ultrasound, pathology slides, etc.).

11. Conclusion

Healthcare does not have to choose between building smarter AI and protecting patient privacy. MedShield-FL proves both goals can be achieved together — hospitals collaborate and build one powerful, shared diagnostic AI model, while every patient's data stays exactly where it should: inside their own hospital, fully private, and fully secure.

By combining Homomorphic Encryption, Vision Transformers, Smart Active Learning, and Explainable AI into one clear, unified system, MedShield-FL offers a practical, understandable, and trustworthy path toward the next generation of privacy-first medical AI.